

Upheap: Upheap is a heap operation used after inserting a new element. The element is repeatedly compared with its parent and swapped upward until the heap-order property is restored. Pseudocode: def upheap(v): while v is not root and v.key < parent(v).key: swap(v, parent(v)); v = parent(v) Time : $O(\log n)$ Space : $O(1)$ Use: Used after insertion to restore the heap-order property in a min-heap.

Downheap: Downheap is a heap operation used after removing the root element. The element is repeatedly compared with its children and swapped with the smaller child until the heap-order property is restored. Pseudocode: def downheap(v): while v has child: small = smaller_child(v); if v.key <= small.key then break; swap(v, small); v = small Time : $O(\log n)$ Space : $O(1)$ Use: Used after removing the root element to restore the heap-order property in a min-heap.

Heap-in-Array Representation: A heap can be stored in an array without using pointers. The root is stored at the first position, and nodes are placed level by level from left to right. Accessing a heap as an array is a complete binary tree. Index Formulas (0-based indexing): Parent(i) = $(i - 1) / 2$, LeftChild(i) = $2i + 1$, RightChild(i) = $2i + 2$. Time : Accessing a node, its parent, or its children using these formulas takes $O(1)$ time. Space : $O(n)$. Use: To implement heaps efficiently in arrays without storing explicit parent or child pointers.

Heap Sort: is a sorting algorithm that inserts all elements into a priority queue (heap) and then repeatedly removes the minimum element to produce a sorted array. Pseudocode: def heap_sort(A): for i = 0 to n-1: pq.insert(A[i]); for i = 0 to n-1: A[i] = pq.removeMin() Time : $O(n \log n)$ Space : $O(n)$ Use: Used to sort elements using a priority queue (heap).

Hash Table is a data structure that stores key-value pairs and uses a hash function to map keys to array indices for fast access. **Hash Function:** A hash function converts a key into an integer called a hash code, which is then mapped to a valid table index using a compression function. **Hash Code:** A hash code is an integer value generated from a key.

Compression Function converts a hash code into a valid table index. A common method is $\text{index} = \text{hashCode} \bmod N$, where N is the table size. Pseudocode: `hashCode = hash(key); index = hashCode mod N` Time : Search $O(1)$, Insert $O(1)$, Delete $O(1)$ on average.

Separate Chaining is a collision resolution technique where each hash table index stores a linked list (chain) of entries that hash to the same index. Operations: get(*k*) retrieves the value associated with key *k* (*index* = hash(*k*) mod *N* → search chain at table[*index*]), put(*k*, *v*) inserts or updates a key-value pair (*index* = hash(*k*) mod *N* → insert/update in chain), delete(*k*) deletes a key-value pair (*index* = hash(*k*) mod *N* → remove key from chain at table[*index*]). Pseudocode: *index* = hash(*k*) mod *N*; perform get/put/delete on the list at table[*index*]. Average Case: Worst-Case: $O(n)$. Note: Separate chaining does not require extra memory for pointers.

Open Addressing handles collisions efficiently and deletion is straightforward. Cons: Requires extra memory for linked lists and performance can degrade to $O(n)$ if many collisions occur.

Probe is a collision resolution technique where, if a hash table position is occupied, the algorithm checks the next positions sequentially until an empty slot or the target key is found. Operations: $get(k)$ retrieves the value associated with key k ($index = hash(k) \bmod N \rightarrow$ while slot occupied and key not found: $index = (index + 1) \bmod N$); $skip$ **DEFUNCT** cells and continue probing; $put(k, v)$ inserts or updates a key-value pair ($index = hash(k) \bmod N \rightarrow$ while slot occupied: $index = (index + 1) \bmod N \rightarrow$ insert); and $remove(k)$ finds the key and marks its position as **DEFUNCT** rather than deleting it completely.

Pseudocode: index = hash(key) mod N; while table[index] is occupied and key not found: index = (index + 1) mod N Time: get (Average: $O(1)$, Worst: $O(n)$), put (Average: $O(1)$, Worst: $O(n)$), remove (Average: $O(1)$, Worst: $O(n)$) Space : $O(n)$. Pros: Simple to implement and does not require extra memory for linked lists. Cons: Suffers from primary clustering, uses DEFUNCT markers after deletion, and performance degrades as the table becomes full. Requires load factor $\alpha < 1$ for good performance; linear probing cannot use fully-filled table

Cuckoo Hashing is a collision resolution technique that uses two hash tables (or two hash functions). Each key can be stored in one of two possible locations. If both locations are occupied, the key is rehashed. **put(k)** inserts the key into one location; if occupied, the existing key is displaced and reinserted into its alternative location. **remove(k)** checks both locations and removes the key if found. **Pseudocode:** **put(k):** place k at $h(k)$; if occupied, evict existing key and place it at its alternative position; repeat until empty slot found or rehash required time $O(1)$ (lookup time and **Worst-Case Clustering**. **Cons:** Insertions can trigger multiple displacements and may require costly rehashing when cycles occur. **Note:** Expected $O(1)$ for **put** and **remove**.

operations only if $N > 2n$ (load factor $\alpha < 0.5$). The **Linear Factor** ($F = n/N$) is defined as the load factor. If $N = n$, where n is the number of stored entries and N is the number of table slots, Impact: A high load factor means the table has more empty slots, resulting in fewer collisions and faster operations. A high load factor means the table is more full, leading to more collisions and slower search, insertion, and deletion operations. As the load factor increases, the performance of collision-resolution methods such as separate chaining and linear probing degrades over time. Hash table operations are typically faster when the load factor is kept below 0.75. For separate chaining, the worst-case approach is $O(n)$ in the worst case when the table becomes too full, requiring a linear scan of all entries. For linear probing, the worst-case approach should be rehashed to maintain efficient performance.

Pigeonhole Principle: If more than n objects are placed into n containers, then at least one container must contain more than one object. Formula: If N objects are placed into k containers and $N > k$, then at least one container contains at least $\lceil N/k \rceil$ objects. Example: If 13 people are placed into 12 months, at least one month must contain the birthdays of at least 2 people. Use: Used to prove that collisions must occur in hash tables when the number of keys exceeds the number of available table slots.

Graph: A graph is a data structure consisting of a set of vertices (nodes) and edges (connections) between those vertices. Graph Terminology: A vertex is a node in the graph, and an edge is a connection between two vertices. A path is a sequence of vertices connected by edges. A simple path is a path that does not repeat any vertex. A cycle is a path that starts and ends at the same vertex. A graph is acyclic if it contains no cycles. Subgraphs: A subgraph is a graph formed from a subset of the vertices and edges of another graph. A simple subgraph contains some of the original vertices and edges. A vertex-induced subgraph contains a chosen set of vertices and all edges between those vertices that

exist in the original graph. An edge-induced subgraph contains a chosen set of edges and all vertices incident to those edges. **Connectivity:** A graph is connected if there exists a path between every pair of vertices. If some vertices cannot reach each other, the graph is disconnected. **Tree, Forest, and Spanning Tree:** A tree is a connected graph with no cycles. A **forest** is a collection of one or more disconnected trees, meaning it is an acyclic graph that may not be connected. A **spanning tree** is a tree formed from a connected graph by deleting a collection of edges so that the resulting graph is connected and contains no cycles. **Acyclic** means containing no cycles. The key difference is that a tree is itself a graph.

Graph Properties: The degree of a vertex is the number of edges incident on it. The Degree Sum Formula states that the sum of the degrees of all vertices in an undirected graph equals $2m$, where m is the number of edges. Formula: $\sum \text{deg}(v) = 2m$ Use: To calculate the number of edges and analyze graph structure.

Edge List: An edge list represents a graph as a list of edges, where each edge stores its two endpoints. Space: $O(n + m)$ Time: Insert Vertex: $O(1)$ Insert Edge: $O(1)$ Search

Adjacency List: An adjacency list represents a graph by storing, for each vertex, a list of its neighboring vertices. Space : $O(n + m)$ Time : Insert Vertex: $O(1)$ Insert Edge: $O(1)$ Search Edge: $O(\deg(v))$ Traversal Graph: $O(n + m)$ Use: Efficient for sparse graphs.

Adjacency Matrix: An adjacency matrix represents a graph using an $n \times n$ matrix, where entry (i, j) is 1 if an edge exists and 0 otherwise. Space : $O(n^2)$ Time : Insert Vertex:

Depth-First Search (DFS): DFS is a graph traversal algorithm that explores as far as possible along one path before backtracking. Pseudocode: def DFS(G): for each vertex v in G: visited[v] = False; parent[v] = None; for each vertex v in G: if not visited[v]: DFS_visit(v) and DFS_visit(v); visited[v] = True; for each adjacent vertex w of v: if not visited[w]: DFS_visit(w). Time Complexity: $O(V + E)$. Space Complexity: $O(V)$.

Breadth-First Search (BFS): BFS is a graph traversal algorithm that explores vertices level by level, visiting all neighbors of a vertex before moving to the next level. Pseudocode: def BFS(G, s): for each vertex v in G: visited[v] = False; parent[v] = None; Q = empty queue; visited[s] = True; enqueue(Q, s); while Q is not empty: v = dequeue(Q); for each

DFS vs BFS: Both (DFS) and (BFS) are graph traversal algorithms with Time $O(n + m)$ and Space $O(n)$, where n is the number of vertices and m is the number of edges. adjacent vertex w of v : if not visited $[w]$: visited $[w] = \text{True}$; parent $[w] = v$; enqueue(Q, w) Time : $O(n + m)$ Space : $O(n)$ Use for graph traversal, building spanning trees, finding connected components, and finding shortest paths in unweighted graphs.

DFS explores one path as deeply as possible before backtracking and uses a stack (or recursion), while BFS explores vertices level by level and uses queue. DFS produces a DFS spanning tree/forest, whereas BFS produces a BFS spanning tree/forest. DFS does not guarantee the shortest path, while BFS finds the shortest path in unweighted graphs. DFS is commonly used for cycle detection, topological sorting, finding connected components, and path finding. BFS is commonly used for shortest path computation in unweighted graphs.

Cut Edge (Bridge) Detection: A cut edge (bridge) is an edge whose removal disconnects the graph. The algorithm uses DFS levels and the `down_up[]` values. Step 1: Compute a DFS tree and level[] for every vertex. Step 2: Compute `down` and `up[]`, the highest ancestor level reachable from `v` using DFS tree edges and at most one back edge. Step 3: A DFS tree edge `(u,v)` is not a bridge if `down_u > level_v`. Otherwise, `(u,v)` is a bridge. Pseudocode: compute DFS tree level[]; compute `down` and `up[]` for

each vertex; for each DFS tree edge (u, v) : if $\text{down_and_up}[v] > \text{level}[u]$ then (u, v) is a bridge Time : $O(n + m)$ Space : $O(n)$ Use: Used to identify critical edges whose removal disconnects the graph. Where: n = number of vertices m = number of edges.

Directed vs Undirected Edge: A directed edge is an ordered pair (u, v) with a direction from the origin (tail) to the destination (head). An undirected edge is an unordered pair $\{u, v\}$ with no direction, meaning the connection works both ways.

Undirected Graph Terminology: In an undirected graph, an edge connects two vertices. A vertex is incident on an edge if it is one of the edge's endpoints. Two vertices are adjacent if they are connected by an edge. The degree of a vertex is the number of incident edges. Parallel edges are multiple edges connecting the same pair of vertices. A self-loop is an edge connecting a vertex to itself. A simple graph contains no parallel edges and no self-loops.

Directed Graph Terminology: In a directed graph, each edge has a tail (origin) and a head (destination). The out-degree of a vertex is the number of outgoing edges, while the in-degree is the number of incoming edges. Parallel edges are multiple directed edges with the same tail and head. A self-loop is an edge from a vertex to itself. Anti-parallel edges are two edges connecting the same vertices in opposite directions, such as (u,v) and (v,u) .

Weighted Graph is a graph in which each edge has an associated numerical value called a weight (or cost), representing quantities such as distance, time, or cost.

Shortest Path Problem: The shortest path problem is the task of finding a path between two vertices whose total edge weight is minimum. **Subpath Property:** A subpath of a shortest path is itself a shortest path. **Shortest Path Tree Property:** There exists a tree of shortest paths from a start vertex to all other reachable vertices. **Dijkstra's Algorithm:** Dijkstra's algorithm finds the shortest paths from a source vertex to all other vertices in a weighted graph with non-negative edge weights. **2.6.3. A* Algorithm:** A* algorithm finds the shortest path from a source vertex to a goal vertex in a weighted graph with non-negative edge weights. **2.6.4. Bellman-Ford Algorithm:** Bellman-Ford algorithm finds the shortest paths from a source vertex to all other vertices in a weighted graph with arbitrary edge weights.

Steps: Set $D[s] = 0$ and $D[v] = \infty$ for all other vertices. Set $\text{parent}[v] = \text{None}$ for all vertices. Insert all vertices into a priority queue using their current distance values. Remove the vertex u with the smallest distance. For each neighbor v of u , check if going through u gives a shorter path. If $D[u] + w(u, v) < D[v]$, update $D[v]$ and $\text{parent}[v] = u$. Update the priority queue with the new distance. Repeat until the priority queue is empty. Pseudocode: `def Dijkstra(G, s):` for each vertex v : $D[v] = \infty$; $\text{parent}[v] = \text{None}$; $D[s] = 0$; $Q \leftarrow$ priority queue containing all vertices $(v, D[v])$; while Q is not empty: $u \leftarrow Q.\text{removeMin}()$; for each edge (u, v) with weight w : if $D[u] + w < D[v]$: $D[v] = D[u] + w$; $\text{parent}[v] = u$; $Q.\text{update}(v, D[v])$.

parent[v] = u; **update**(Q, v, D[v]) Time : $O(m \log n)$ using a heap. Space : $O(n)$
Use: To find shortest paths from a source vertex to all other vertices in a weighted graph with non-negative edge weights. **Important Requirement:** All edge weights must be non-negative. n = number of vertices m = number of edges.
Relaxation: Relaxation is the key operation in Dijkstra's algorithm. It checks whether a shorter path to a vertex can be found through another vertex and updates the distance

Minimum Spanning Tree (MST): A Minimum Spanning Tree (MST) is a spanning tree of a connected weighted graph whose total edge weight is minimum among all possible

Cut Property of MST: Let S be any subset of vertices, and let e be the minimum-weight edge crossing the cut between S and $V - S$. Then e must belong to the MST. Use:

Cycle Property of MST: Let C be any cycle in a graph, and let e be the maximum-weight edge in that cycle. Then e cannot belong to the MST. Use: Used to eliminate unnecessary edges and forms the basis of Kruskal's algorithm.

a vertex in the tree to a vertex outside the tree. Steps: Choose any start vertex s . Set $D[s] = 0$ and $D[v] = \infty$ for all other vertices. Set $\text{parent}[v] = \text{None}$ for all vertices. Insert all vertices into a priority queue using their current D values. Remove the vertex u with the smallest key from the priority queue. For each adjacent vertex v , if v is still in the queue and $w(u,v) < D[v]$, update $D[v]$ and set $\text{parent}[v] = u$. Continue until the priority queue is empty. The edges $(\text{parent}[v], v)$ form the MST. Pseudocode: $\text{def Prim}(G, s)$:

Kruskal's Algorithm: Kruskal's algorithm finds a Minimum Spanning Tree (MST) by repeatedly selecting the smallest-weight edge that does not create a cycle. For each vertex v : $D[v] = 0$; $\text{parent}[v] = \text{No}$; $D[s] = 0$; $Q \leftarrow$ priority queue containing all vertices $(v, D[v])$; while Q is not empty: $u = Q.\text{removeMin}()$; for each edge (u, v) with weight w : if v in V and $w < D[v]$: $D[v] = w$; $\text{parent}[v] = u$; $\text{update}(Q, v, D[v])$; return parent Time: $O(m \log n)$ using a heap, using a Fibonacci heap: $O(m + n \log n)$. Space: $O(n)$ Use: To find a Minimum Spanning Tree of a connected weighted graph. Where: n = number of vertices m = number of edges.

Steps: Create a separate set for each vertex. Sort all edges in non-decreasing order of weight. Process edges from smallest to largest. For each edge (u,v) , check whether u and v belong to different sets. If they belong to different sets, add the edge to the MST and merge the two sets. If they belong to the same set, ignore the edge because it would create a cycle. Stop when the MST contains $n - 1$ edges. Pseudocode: `def Kruskal(G): T = empty set; for each vertex v: makeSet(v); sort edges by increasing weight; for each edge e in E: if find(u) != find(v): T.add(e); v1, u1 = union(e); with activated v, makeSet(u1);`

Greedy Algorithm: A greedy algorithm is an algorithmic technique that builds a solution step by step by always choosing the locally optimal choice at each step, hoping that these choices lead to a globally optimal solution. Greedy Framework: Start with an empty solution. Repeatedly choose the best available option according to a greedy criterion.

Add the chosen option to the solution if it is feasible. Never reconsider or undo previous choices. Continue until a complete solution is obtained. Pseudocode: solution = $\emptyset$; while solution not complete; choose best candidate; if feasible: add candidate to solution; return solution Use: Used in algorithms such as Kruskal's MST, Prim's MST, Dijkstra's shortest path, Huffman coding, and activity selection.

Fractional Knapsack: The Fractional Knapsack problem aims to maximize the total value of items placed in a knapsack of capacity W , where fractions of items may be taken. Steps: Compute the value-to-weight ratio for each item. Sort items in decreasing order of ratio. Repeatedly take the item with the highest ratio. If the whole item fits, take it completely. Otherwise, take only the fraction that fits and stop. Pseudocode: sort items by decreasing value/weight ratio; capacity_remaining = W ; for each item: if capacity_remaining == 0: break; if item.weight <= capacity_remaining: take whole item; capacity_remaining -= item.weight; else: take fraction = capacity_remaining / item.weight; break; return total_value Time : $O(n \log n)$ (due to sorting) Space : $O(1)$ (excluding the input array and sorting space) Use: Used to maximize value when fractional amounts of items can be selected.

Fractional Knapsack: The Fractional Knapsack problem aims to maximize the total value of items placed in a knapsack of capacity W , where fractions of items may be taken. Steps: Compute the value-to-weight ratio for each item. Sort items in decreasing order of ratio. Repeatedly take the item with the highest ratio. If the whole item fits, take it completely. Otherwise, take only the fraction that fits and stop. Pseudocode: sort items by decreasing value/weight ratio; capacity_remaining = W ; for each item: if capacity_remaining == 0: break; if item.weight <= capacity_remaining: take whole item; capacity_remaining -= item.weight; else: take fraction = capacity_remaining / item.weight; break; return total_value Time : $O(n \log n)$ (due to sorting) Space : $O(1)$ (excluding the input array and sorting space) Use: Used to maximize value when fractional amounts of items can be selected.